



Tunisian Republic
Ministry of Higher Education and Scientific
Research
University of Sousse
**Higher Institute of Computer Science
and Communication Technologies
(ISITCOM)**



End of Studies Project Report

Submitted in partial fulfillment of the requirements for the
National Engineering Degree — Specialty: Teleinformatics

Presented by:

Asma Daoussi

Development of a Digital Asset Marketplace

for Inherited Games Studio (IGS)

Carried out at: Inherited Games Studio (IGS), Tunisia

Academic Supervisor: Mr. Sami Ben Amor

Company Supervisor: Mme. Nour Ltaief

Defended on /06/2026 **before the jury composed of:**

Jury President: Mr. _____

Rapporteur: Mr. _____

Academic Year: 2025–2026

Authorization for Defense

Company Supervisor

I hereby authorize the student to submit their End of Studies Project Report for the jury defense.

Mrs Nour LTAIEF

Company Supervisor, Inherited Games Studio

Signature: _____

Academic Supervisor

I hereby authorize the student to submit their End of Studies Project Report for the jury defense.

Mr Sami BEN AMOR

Academic Supervisor, ISITCOM — University of Sousse

Signature: _____

Dedication

To my beloved family,

*who have been my unwavering pillar of strength
throughout every step of this journey.*

To my parents,

*whose sacrifices, endless encouragement, and unconditional love
have been the foundation upon which I built every dream.*

To my brothers and sisters,

*for the laughter, the support, and the belief
that always pushed me forward.*

To my friends and colleagues,

*who made these long study years memorable
and filled them with moments of joy and solidarity.*

*To every person who believed in me
even when I doubted myself —*

this work is for you.

Asma Daoussi

Acknowledgments

First and foremost, I would like to express my sincere gratitude to **Mr. Sami Ben Amor**, my academic supervisor, for his continuous guidance, insightful advice, and unwavering support throughout the realization of this project. His expertise and constructive feedback have been invaluable in shaping both the technical direction and the quality of this work.

I extend my heartfelt thanks to **Mme. Nour Ltaief**, my company supervisor at Inherited Games Studio, for welcoming me into the team, sharing her professional experience, and providing the resources and environment necessary to bring this project to life. Her trust in my capabilities and her constant encouragement made this internship an enriching and rewarding experience.

I would like to express my appreciation to all the staff at **Inherited Games Studio (IGS)** for their warm welcome and their readiness to assist me whenever needed. Working alongside such a talented and passionate team has been a privilege.

My gratitude also goes to the faculty and staff of **ISITCOM — Higher Institute of Computer Science and Communication Technologies** at the **University of Sousse**, whose dedication to quality education provided me with the theoretical and practical foundations that made this project possible.

I am also deeply thankful to the members of the **jury** for dedicating their time to evaluate this work, and for the honor they grant me by participating in its defense.

Finally, I would like to thank my family and friends for their patience, encouragement, and moral support during the long nights and challenging moments of this project.

Asma Daoussi
Sousse, 2026

Contents

Dedication	i
Acknowledgments	ii
List of Figures	vi
List of Tables	vii
List of Acronyms	viii
General Introduction	1
1 General Context	2
1.1 Host Organization: Inherited Games Studio	2
1.2 Problem Statement	2
1.3 Comparative Study of Existing Solutions	3
1.4 Proposed Solution	4
1.5 Development Methodology: Scrum	4
1.6 Technology Stack	6
2 Requirements Analysis	7
2.1 Actors Identification	7
2.2 Functional Requirements	8
2.3 Non-Functional Requirements	9
2.4 Global Use Case Diagram	10
2.5 System Architecture	10
3 Sprint 0: Research, Planning & Architecture	12
3.1 System Architecture	12
3.2 Technology Stack Justification	13
3.3 Database Design	13
4 Sprints 1–2: Core Platform	15
4.1 Sprint 1: Platform Foundation	15
4.1.1 Sprint Backlog	15
4.1.2 Analysis and Design	16

4.1.3	Sprint Review — Deliverables	18
4.2	Sprint 2: Marketplace Core Features	18
4.2.1	Sprint Backlog	18
4.2.2	Analysis and Design	19
4.2.3	Sprint Review — Deliverables	22
5	Sprints 3–4: Marketplace Experience	23
5.1	Sprint 3: Content Management and Administration	23
5.1.1	Sprint Backlog	23
5.1.2	Analysis and Design	24
5.1.3	Sprint Review — Deliverables	26
5.2	Sprint 4: Role-Based Workflows and Community Features	27
5.2.1	Sprint Backlog	27
5.2.2	Analysis and Design	28
5.2.3	Sprint Review — Deliverables	29
6	Sprints 5–6: Advanced Features	30
6.1	Sprint 5: Internal Collaboration Platform	30
6.1.1	Sprint Backlog	30
6.1.2	Analysis and Design	31
6.1.3	Sprint Review — Deliverables	34
6.2	Sprint 6: Artificial Intelligence Integration	35
6.2.1	Sprint Backlog	35
6.2.2	AI System Architecture	35
6.2.3	Model Training and Evaluation	37
6.2.4	AI Review Analysis	37
6.2.5	Sprint Review — Deliverables	38
7	Sprints 7–8: IGS Workspace & Admin Dashboard	39
7.1	Sprint 7: AI-Enhanced Services and Catalog Expansion	39
7.1.1	Sprint Backlog	39
7.1.2	AI Recommendation Engine	40
7.1.3	IGS Marketplace Bot	41
7.1.4	Catalog Expansion	41
7.1.5	Seed Data	41
7.1.6	Sprint Review — Deliverables	41
7.2	Sprint 8: Admin Dashboard & Premium UI Overhaul	42
7.2.1	Sprint Backlog	42
7.2.2	Implementation	43
7.2.3	Sprint Review — Deliverables	43

8	CI/CD & Deployment	45
8.1	CI/CD Pipeline	45
8.1.1	Deployment Pipeline	45
8.2	Containerization with Docker	46
8.3	Nginx Reverse Proxy	46
8.4	Deployment Strategy	46
8.4.1	Image Publication and Remote Update	46
8.4.2	Database Migrations	47
8.4.3	Environment Secrets	47
8.5	API Documentation	47
8.6	Testing Strategy	48
	General Conclusion	49
	Bibliography	50
	Webography	51
	Abstract	52

List of Figures

1.1	Global Architecture of the IGS Platform	4
2.1	Global Use Case Diagram	10
2.2	Backend Layered Architecture — Routes → Controllers → Repositories	11
4.1	Authentication Flow — JWT + Firebase (Sequence Diagram)	17
4.2	Sprint 1 — RBAC Role Hierarchy	17
4.3	Sprint 2 — Asset and Order Entity-Relationship Diagram	20
4.4	Stripe Payment Flow (Sequence Diagram)	21
5.1	Sprint 3 — Reviews, Notifications and Subscriptions ERD	25
5.2	Admin Moderation Use Case Diagram	26
5.3	Intern Asset Request Workflow (Sequence Diagram)	28
5.4	Sprint 4 — Entity-Relationship Diagram	29
6.1	IGS Workspace Architecture and Integration with Marketplace	32
6.2	Real-Time Chat Architecture with Socket.io	33
6.3	Sprint 5 — Workspace and Chat Entity-Relationship Diagram	34
6.4	IGS Bot AI Architecture — Trained ML Models + Open-Source LLM .	36
8.1	IGS Marketplace Deployment Pipeline	45

List of Tables

1.1	Inherited Games Studio — Company Profile	2
1.2	Comparative Analysis of Existing 3D Asset Marketplace Platforms . .	3
1.3	Sprint Planning Overview	5
1.4	Technology Stack Summary	6
2.1	System Actors and Permission Levels	7
2.2	Functional Requirements Summary	8
2.3	Non-Functional Requirements	9
3.1	Sprint 0 — Technology Stack	13
4.1	Sprint 1 — User Stories	16
4.2	Sprint 2 — User Stories	19
4.3	Asset Categories and Viewer Components	22
5.1	Sprint 3 — User Stories	24
5.2	Subscription Plan Tiers	25
5.3	Sprint 4 — User Stories	27
6.1	Sprint 5 — User Stories	31
6.2	Workspace Sidebar Navigation by Role	32
6.3	Sprint 6 — User Stories	35
6.4	AI Components and Models Used	37
6.5	Training Datasets	37
7.1	Sprint 7 — User Stories	40
7.2	New Product Categories Added in Sprint 7	41
7.3	Sprint 8 — User Stories	43
8.1	GitHub Actions Secrets — Classification	47
8.2	API Documentation — Collections by Domain	48
8.3	Test Coverage by Layer	48

List of Acronyms

Acronym	Definition
API	Application Programming Interface
CDN	Content Delivery Network
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
DB	Database
DM	Direct Message
ERP	Enterprise Resource Planning
GLB	GL Binary (3D format)
GLTF	GL Transmission Format
HDRI	High Dynamic Range Imaging
HTTP	HyperText Transfer Protocol
IGS	Inherited Games Studio
ISITCOM	Institut Supérieur de l'Informatique et des Technologies de Communication
JWT	JSON Web Token
MVC	Model-View-Controller
OAuth	Open Authorization
ORM	Object-Relational Mapping
PBR	Physically Based Rendering
PFE	Projet de Fin d'Études (End of Studies Project)
RBAC	Role-Based Access Control
REST	Representational State Transfer
SPA	Single Page Application
SQL	Structured Query Language
SSO	Single Sign-On
UML	Unified Modeling Language
UUID	Universally Unique Identifier

General Introduction

The global digital content economy is experiencing remarkable expansion, with the 3D model marketplace market valued at **USD 1.92 billion** in 2023 and projected to reach **USD 4.86 billion by 2031** (CAGR of 11.2%). This growth is driven by the booming video game industry, architectural visualization, and film production. Despite this global trend, **Tunisia had no dedicated 3D digital asset marketplace** before this project — Tunisian studios were forced to rely entirely on foreign platforms imposing dollar-denominated pricing, foreign payment gateways, and no customization.

Inherited Games Studio (IGS), a Tunisian video game development company, commissioned the development of **IGS Marketplace** — the first dedicated 3D digital asset marketplace designed and built in Tunisia. The platform serves three distinct user communities: external creators and buyers needing a seamless marketplace experience; internal employees and interns requiring role-based access to studio tools; and administrators demanding full control over content, users, and platform analytics.

This end-of-studies project was carried out as a six-month internship within IGS. The result is a full-stack web platform composed of two interconnected applications: **IGS Marketplace**, the public-facing marketplace with an interactive 3D viewer, Stripe payment, AI services, and subscription plans; and **IGS Workspace**, a companion internal dashboard for team management, task tracking, and real-time collaboration. Both applications share the same Express.js backend, PostgreSQL database, and Firebase authentication layer.

This report is organized following the **Scrum** agile methodology. Chapter 1 establishes the project context; Chapter 2 covers the requirements analysis; Chapters 3 through 7 each document a sprint or pair of sprints; and Chapter 8 presents the CI/CD and deployment strategy.

Chapter 1

General Context

Introduction

This chapter presents the host organization, the problem statement, a comparative study of existing solutions, the proposed solution, the chosen development methodology, and the technology stack.

1.1 Host Organization: Inherited Games Studio

Inherited Games Studio (IGS) is a Tunisian independent video game development company specializing in narrative-driven and multiplayer games. The studio brings together artists, developers, designers, and sound engineers focused on immersive interactive experiences.

Table 1.1: Inherited Games Studio — Company Profile

Field	Information
Industry	Video Game Development / Digital Content
Location	Tunisia
Specialization	3D game development, digital assets, interactive media
Company Supervisor	Mme. Nour Ltaief

1.2 Problem Statement

Before this project, IGS faced several critical operational pain points:

- **No centralized asset repository:** Assets were scattered across local drives with no unified catalog or search.
- **No monetization platform:** IGS had no mechanism to sell surplus assets externally.

- **No role-based access control:** All internal users shared similar access levels, creating security concerns.
- **Fragmented workflows:** Collaboration relied on disconnected tools with no unified project management.
- **Dependency on foreign platforms:** International platforms (Unity Asset Store, Sketchfab, Fab) imposed dollar-denominated fees and no customization.

1.3 Comparative Study of Existing Solutions

The following table compares the main international 3D asset platforms against IGS Marketplace across the features critical to the studio's needs:

Table 1.2: Comparative Analysis of Existing 3D Asset Marketplace Platforms

Feature	Unity AS	Fab	Sketchfab	CGTrader	Gumroad	IGS MP
3D In-Browser Viewer	Partial	Yes	Yes	No	No	Yes
Multi-Engine Support	No	Yes	Yes	Yes	Yes	Yes
Custom RBAC	No	No	No	No	No	Yes
Internal Team Tools	No	No	No	No	No	Yes
AI Features	No	No	No	No	No	Yes
Self-Hosted / Custom	No	No	No	No	No	Yes
Intern Workflow	No	No	No	No	No	Yes
Real-Time Chat	No	No	No	No	No	Yes
Stripe Payment	No	No	No	No	Yes	Yes
Tunisia-Accessible	Partial	Partial	Partial	Partial	Yes	Yes

No existing platform combines a 3D digital asset marketplace with internal studio management tools, customizable role hierarchies, and AI-powered features in a single self-hosted solution. IGS Marketplace is therefore not merely a custom tool but the **first 3D digital asset marketplace platform designed and developed in Tunisia**.

1.4 Proposed Solution

The proposed solution is a full-stack web platform composed of two interconnected applications:

1. **IGS Marketplace (port 5173):** The public-facing marketplace where sellers list assets, buyers purchase and download content, and administrators moderate the catalog. It includes a 3D interactive viewer, Stripe checkout, a subscription system, and an AI chatbot assistant.
2. **IGS Workspace (port 5174):** A private internal dashboard exclusively for IGS employees and interns, featuring team management, task tracking, project ERP modules, and a real-time direct messaging system.

Both applications share the same backend API (Express.js on port 3001), the same PostgreSQL database, and the same authentication system (Firebase Auth + JWT), enabling seamless Single Sign-On between the two apps.

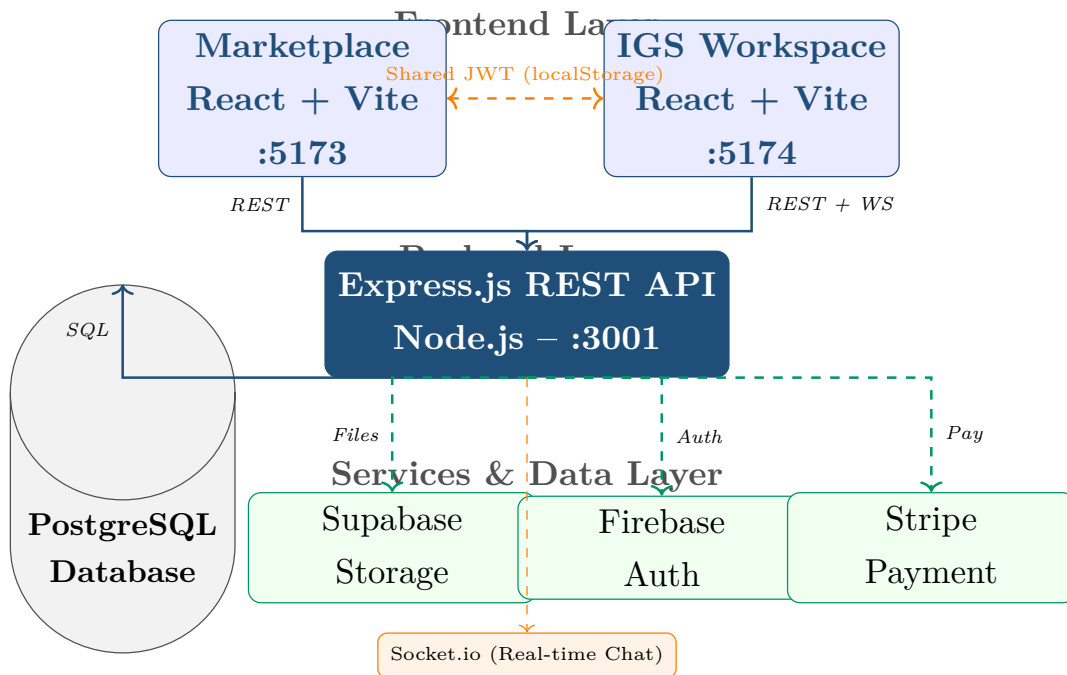


Figure 1.1: Global Architecture of the IGS Platform

1.5 Development Methodology: Scrum

Scrum was selected as the development methodology because requirements emerged progressively throughout the internship, the company supervisor (Mme. Nour Ltaief) naturally assumes the Product Owner role, the academic supervisor (Mr. Sami Ben

Amor) assumes the Scrum Master role, and sprint reviews align with mandatory internship progress meetings.

The project was divided into **eight sprints**, each lasting approximately two to three weeks:

Table 1.3: Sprint Planning Overview

Sprint	Theme	Key Deliverables
0	Research & Architecture	Architecture design, DB schema, environment setup
1	Core Infrastructure	Backend, PostgreSQL, JWT auth, RBAC (6 roles)
2	Asset Marketplace Core	Catalog, 3D viewer, seller upload, cart, Stripe
3	Content & Administration	Reviews, notifications, admin panel, subscriptions
4	Advanced Role Features	Intern workflow, seller analytics, trading, groups
5	IGS Workspace	Companion app, task management, real-time chat
6	AI Integration	ML models, sentiment analysis, chatbot
7 & 8	AI Expansion & Polish	Recommendation engine, UI overhaul, admin dashboard

1.6 Technology Stack

Table 1.4: Technology Stack Summary

Layer	Technology	Role / Justification
Frontend	React 18 + Vite	SPA framework — fast HMR, component model
	Three.js / Model Viewer	3D asset viewer in the browser
	TailwindCSS	Utility-first responsive styling
Backend	Node.js + Express.js	REST API — non-blocking, JSON-native
	Socket.io	WebSocket real-time events (chat, notifications)
	HuggingFace / Mistral-7B	AI review analysis, chatbot, recommendations
Data & Auth	PostgreSQL 16	Primary relational database
	Supabase Storage	Scalable file hosting for 3D assets
	Firebase Auth	Google OAuth2 + email/password authentication
Payments	Stripe	Secure card payment processing
Dev Tools	Git + GitHub	Version control and CI/CD
	Postman	API endpoint testing

Conclusion

This chapter established the project context: the host organization IGS and its operational challenges, the comparative analysis confirming the absence of a dedicated 3D marketplace in Tunisia, the proposed two-application solution, the Scrum methodology applied across eight sprints, and the technology stack. The following chapter details the functional and non-functional requirements of the system.

Chapter 2

Requirements Analysis

Introduction

This chapter identifies the system actors, enumerates functional and non-functional requirements, presents the global use case diagram, and defines the system architecture.

2.1 Actors Identification

The IGS Marketplace platform serves six distinct actor types organized in a hierarchical RBAC system:

Table 2.1: System Actors and Permission Levels

Role	Type	Level	Key Permissions
super_admin	Internal	100	Full system access, manage roles, system settings
admin	Internal	80	Manage users, assets, orders, view analytics
employee	Internal	60	Moderate content, view all analytics
intern	Internal	40	Browse catalog, request free assets, view tasks
seller	External	20	Upload assets, manage own listings, view analytics
member	External	10	Purchase assets, add to cart, write reviews

2.2 Functional Requirements

Table 2.2: Functional Requirements Summary

Domain	Requirements
Authentication	Register via email/password or Google OAuth2; JWT-based API authentication; profile management; admin role management.
Asset Catalog	Browse, filter (category, price, rating, free/paid), and search assets; 13 categories; interactive 3D/HDRI/texture viewer.
Seller Upload	Multi-format upload (GLB, FBX, OBJ, PNG, HDR...); pending moderation workflow; edit request after approval; seller analytics dashboard.
Shopping & Orders	Persistent cart; Stripe payment; order tracking; automatic library unlock; free asset claiming.
Reviews & Ratings	Ownership-gated review submission; 1–5 star rating; AI sentiment analysis; automatic moderation flagging.
Admin Panel	Approve/reject assets with written reason; manage users and roles; platform-wide analytics; audit trail.
Notifications	Real-time and persistent in-app notifications; unread badge; mark as read.
Intern Workflow	Asset request with justification; voucher generation; supervised upload.
Community	P2P asset trading; community groups; seller role request flow.
IGS Workspace	Internal app: team management, task tracking, project ERP, real-time chat (Socket.io), role-based navigation.
AI Services	Personalized recommendation engine; marketplace chatbot (Mistral-7B); automated review sentiment analysis.

2.3 Non-Functional Requirements

Table 2.3: Non-Functional Requirements

Category	Requirements
Performance	API response under 300 ms for standard queries; asset files streamed to cloud storage; queries indexed.
Security	JWT expiration; Firebase token verification on all protected routes; RBAC middleware; parameterized queries; CORS; HTTPS in production.
Scalability	Stateless REST API; cloud file storage (Supabase); connection pooling; horizontal scaling ready.
Usability	Responsive UI for desktop and tablet; loading states and error messages on all user-facing operations.
Reliability	Centralized error handling; database transactions for orders; audit logging for admin actions.
Maintainability	Routes → Controllers → Repositories separation; modular React components; consistent naming conventions.

2.4 Global Use Case Diagram

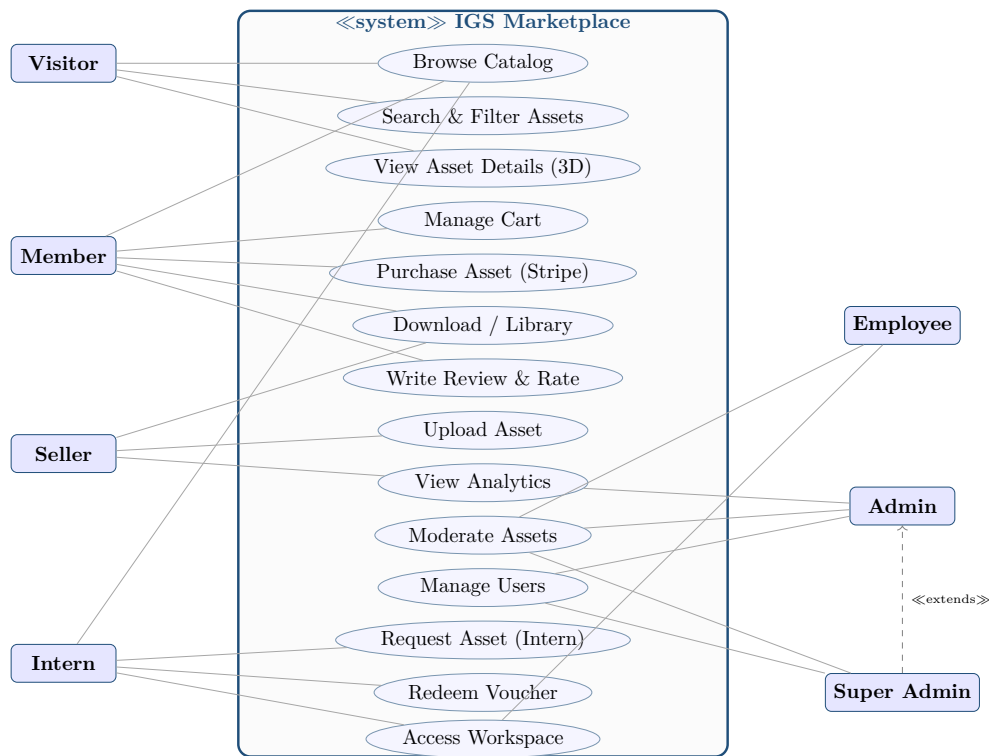


Figure 2.1: Global Use Case Diagram

2.5 System Architecture

The IGS platform follows a **three-tier distributed architecture** separating presentation, business logic, and data access:

- **Tier 1 — Presentation:** Two independent React + Vite SPAs (Marketplace on port 5173 and IGS Workspace on port 5174), communicating with the backend exclusively via REST and sharing a JWT stored in `localStorage`.
- **Tier 2 — Business Logic:** A single Express.js REST API (port 3001) handling authentication, authorization, domain logic, and external service orchestration (Stripe, Supabase, Firebase, AI).
- **Tier 3 — Data:** A PostgreSQL database (30+ tables) for relational persistence, and Supabase Storage for binary asset files.

Within Tier 2, the backend follows the **Repository pattern** — each entity has its own repository file encapsulating all SQL queries. Controllers never write raw SQL; they always go through a repository method. This decouples business logic from database access and makes the data layer independently testable.

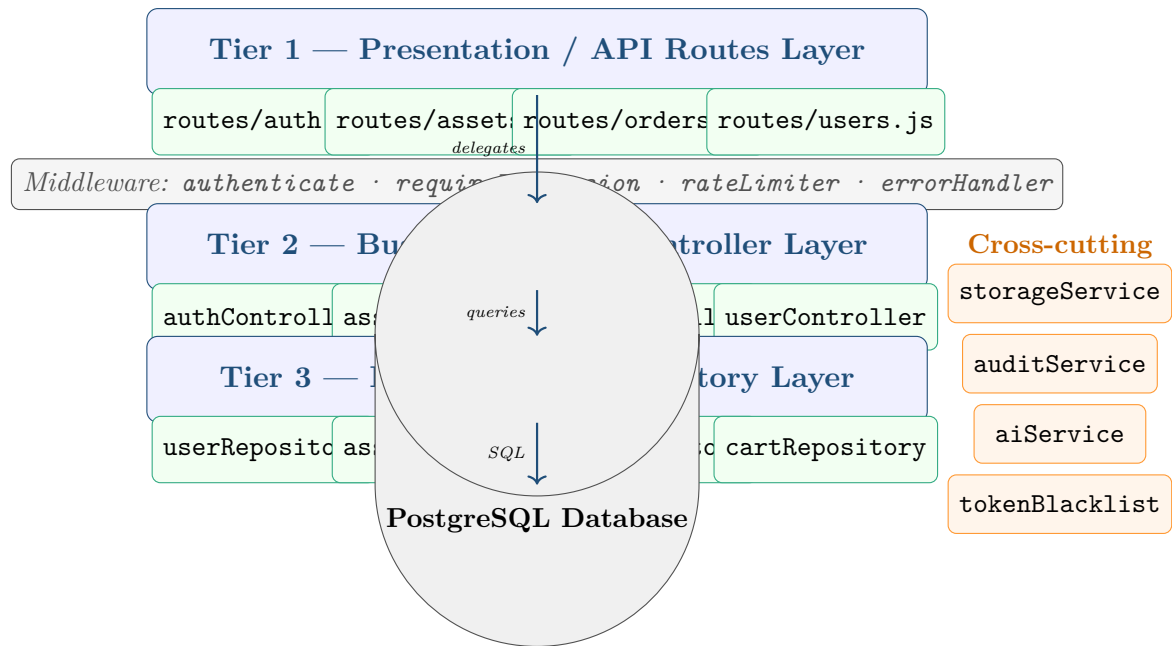


Figure 2.2: Backend Layered Architecture — Routes → Controllers → Repositories

Conclusion

This chapter identified all system actors, detailed functional and non-functional requirements, and established the architectural foundation. The platform follows a three-tier distributed architecture with the Repository pattern on the backend. The following chapters each cover one sprint or pair of sprints of the development cycle.

Chapter 3

Sprint 0: Research, Planning & Architecture

Introduction

Sprint 0 constitutes the foundational preparation phase before any code is written. It encompasses market research, requirements elicitation, architectural decision-making, technology selection, and database design. The deliverables of Sprint 0 are the design artefacts and infrastructure choices that guide all subsequent development sprints.

3.1 System Architecture

IGS Marketplace follows a **three-tier architecture** separating presentation, business logic, and data concerns. The two React + Vite SPAs (ports 5173 and 5174) communicate exclusively via a REST API with the Node.js/Express backend (port 3001), which in turn manages a PostgreSQL relational database and Supabase Storage for binary files.

The backend is internally structured around the **Routes → Controllers → Repositories** layered pattern with cross-cutting services (`storageService`, `auditService`, `aiService`, `tokenBlacklist`) used across multiple controllers.

3.2 Technology Stack Justification

Table 3.1: Sprint 0 — Technology Stack

Layer	Technology	Justification
Frontend	React 18 + Vite	Component model, rich ecosystem, fast HMR
3D Rendering	Three.js + Model Viewer	WebGL abstraction, native GLB/GLTF support
Backend	Node.js 20 + Express 4	Non-blocking I/O, unified JavaScript stack
Database	PostgreSQL 16	ACID compliance, JSONB columns, strong indexing
Authentication	Firebase Auth	Google OAuth2 out-of-the-box, tested JWT flow
File Storage	Supabase Storage	S3-compatible CDN, generous free tier
Payments	Stripe	International cards, PaymentIntent API
Real-time	Socket.io v4	Bidirectional events, room-based broadcasting
AI / LLM	HuggingFace Inference	Free-tier access to Mistral-7B open-source LLM

3.3 Database Design

The complete database schema was designed in Sprint 0 and implemented incrementally through 18 numbered migration scripts. The schema is organized into six conceptual groups:

1. **Identity & Access:** `users`, `user_roles`, `role_permissions`, `audit_logs`
2. **Catalog:** `assets`, `categories`, `asset_tags`, `asset_formats`
3. **Commerce:** `orders`, `order_items`, `cart_items`, `user_downloads`, `subscription_plans`, `user_subscriptions`
4. **Community:** `reviews`, `asset_likes`, `notifications`, `groups`, `group_members`, `trades`

5. **Workspace:** workspace_tasks, projects, chat_channels, chat_messages, direct_messages
6. **AI & Events:** user_events, intern_asset_requests, intern_vouchers

Conclusion

Sprint 0 established the architectural blueprint, technology choices, and database design on which the entire platform is built. The schema and project structure defined here were adhered to consistently throughout all subsequent sprints, minimizing rework and enabling rapid iterative delivery.

Chapter 4

Sprints 1–2: Core Platform

Introduction

Chapter 4 covers the first two development sprints, which together deliver the entire core technical infrastructure of IGS Marketplace. Sprint 1 establishes the backend foundation — authentication, Role-Based Access Control, and the PostgreSQL database schema. Sprint 2 builds directly upon it to deliver the end-user-facing marketplace: the asset catalog, interactive 3D viewer, persistent shopping cart, and Stripe payment integration.

4.1 Sprint 1: Platform Foundation

Introduction

Sprint 1 establishes the technical foundation of the platform: a fully operational Express.js server with PostgreSQL, a complete authentication system (JWT + OAuth2), and a six-level Role-Based Access Control layer. The React frontend skeleton is also initialized with authentication and permissions contexts.

4.1.1 Sprint Backlog

Sprint 1 — Duration: 3 weeks

Goal: Deliver a fully functional authentication system with RBAC on a clean backend and database foundation.

Table 4.1: Sprint 1 — User Stories

ID	User Story	Priority	Points
US-01	As a user, I want to register with email and password.	High	3
US-02	As a user, I want to log in with Google (OAuth2).	High	5
US-03	As a user, I want to receive a JWT for API authentication.	High	3
US-04	As an admin, I want every endpoint protected by role-based permissions.	High	8
US-05	As a developer, I need a PostgreSQL schema for all platform data.	High	5
US-06	As a user, I want to view and edit my profile.	Medium	3
US-07	As an admin, I want all user actions recorded in an audit trail.	Medium	3
Total Story Points:			30

4.1.2 Analysis and Design

Authentication Flow

The authentication flow operates as follows: the client authenticates with Firebase (email/password or Google OAuth2); Firebase returns a signed ID token; the client sends the token in every subsequent API request; the backend middleware verifies it via the Firebase Admin SDK; and if the user does not yet exist in PostgreSQL, a record is automatically created with the default *member* role.

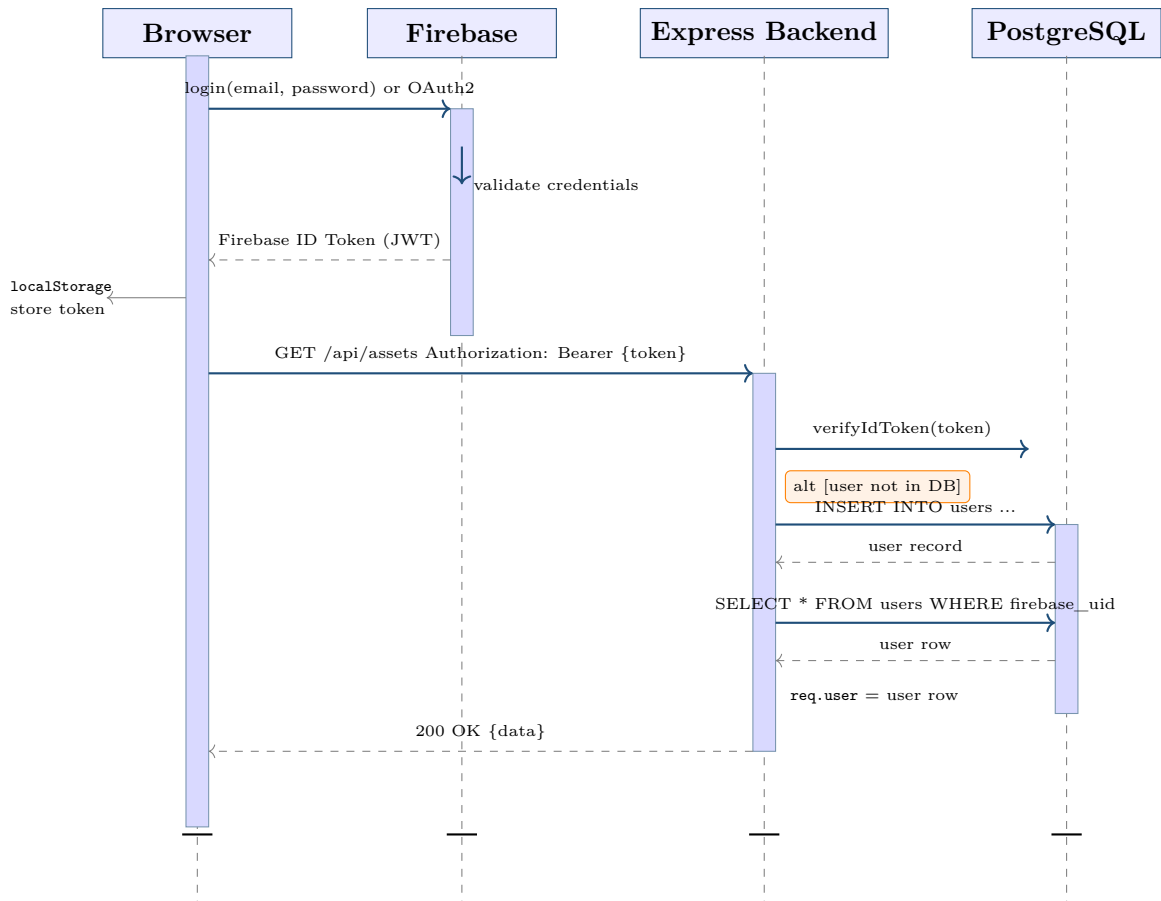


Figure 4.1: Authentication Flow — JWT + Firebase (Sequence Diagram)

RBAC Architecture

The permission system is built on two database tables: `user_roles` and `role_permissions`. A custom middleware checks whether the authenticated user’s role holds the required permission before granting access to any protected route. The six roles are ordered by privilege level: *super_admin* (100) > *admin* (80) > *employee* (60) > *intern* (40) > *seller* (20) > *member* (10).

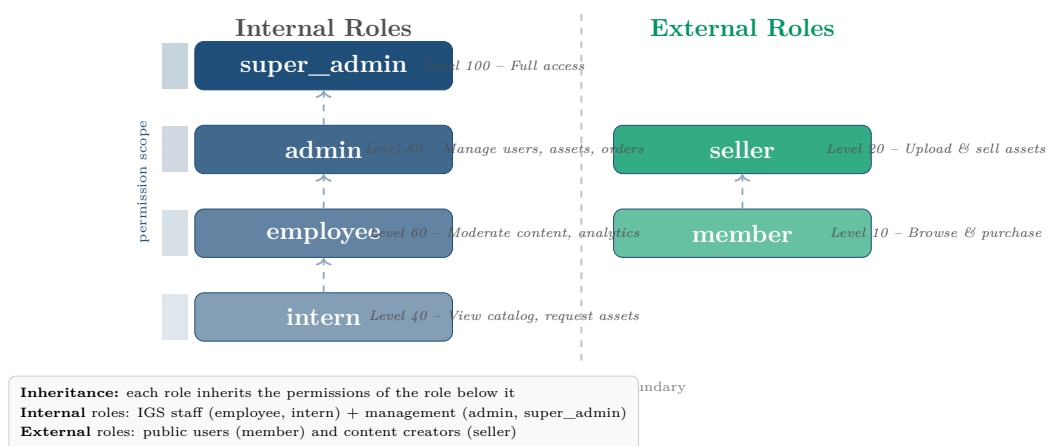


Figure 4.2: Sprint 1 — RBAC Role Hierarchy

4.1.3 Sprint Review — Deliverables

- Express.js server with full middleware stack (CORS, body parser, error handler, audit logger).
- PostgreSQL schema with core tables (`users`, `categories`, `audit_logs`) and RBAC permission tables.
- Email/password and Google OAuth2 authentication flows with Firebase Admin SDK.
- RBAC middleware protecting all sensitive API endpoints.
- React frontend with `AuthContext`, `PermissionsContext`, and `ProtectedRoute` component.
- User profile page with view and edit capability.

Conclusion

Sprint 1 delivered the foundational infrastructure on which all subsequent features are built. The authentication system, RBAC layer, and database schema are stable and extensible.

4.2 Sprint 2: Marketplace Core Features

Introduction

Sprint 2 delivers the heart of the platform: the asset catalog, interactive 3D viewer, seller upload workflow, shopping cart, and Stripe payment integration. By the end of this sprint, users can browse, purchase, and download digital assets through a complete end-to-end e-commerce flow.

4.2.1 Sprint Backlog

Sprint 2 — Duration: 3 weeks

Goal: Deliver a functional end-to-end marketplace — from asset listing to purchase and download.

Table 4.2: Sprint 2 — User Stories

ID	User Story	Priority	Points
US-08	As a visitor, I want to browse the catalog with filters.	High	5
US-09	As a visitor, I want an asset detail page with a 3D preview.	High	8
US-10	As a seller, I want to upload an asset with files and metadata.	High	8
US-11	As a member, I want to add assets to my cart.	High	3
US-12	As a member, I want to pay securely with Stripe.	High	8
US-13	As a member, I want to access my library to re-download owned assets.	High	3
US-14	As a member, I want to claim free assets without payment.	Medium	2
US-15	As a seller, I want to upload multiple file formats per asset.	Medium	5
Total Story Points:			42

4.2.2 Analysis and Design

Database Schema

Sprint 2 extends the database with asset and commerce tables: `assets` (with a JSONB `metadata` column for category-specific attributes), `asset_tags`, `asset_formats` (multiple downloadable formats per asset), `orders`, `order_items`, `cart_items` (stored in PostgreSQL for cross-device persistence), and `user_downloads` (ownership records).

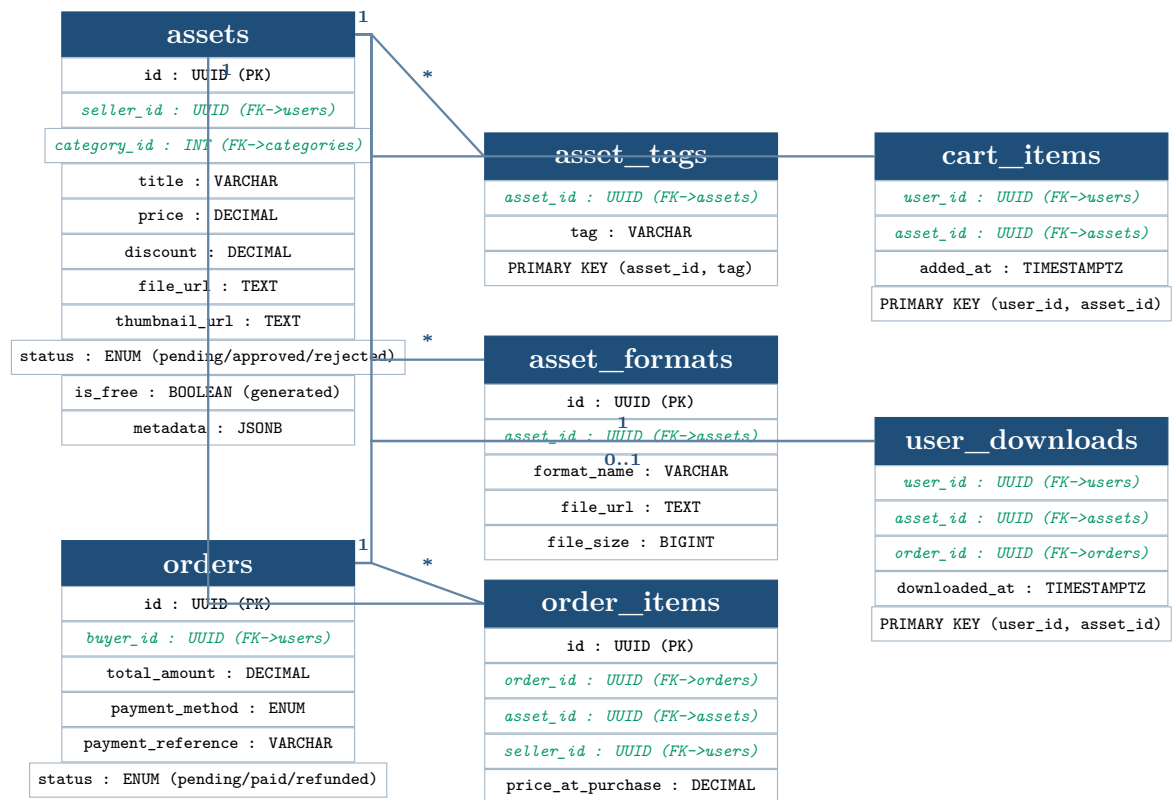


Figure 4.3: Sprint 2 — Asset and Order Entity-Relationship Diagram

Payment Flow — Stripe PaymentIntent

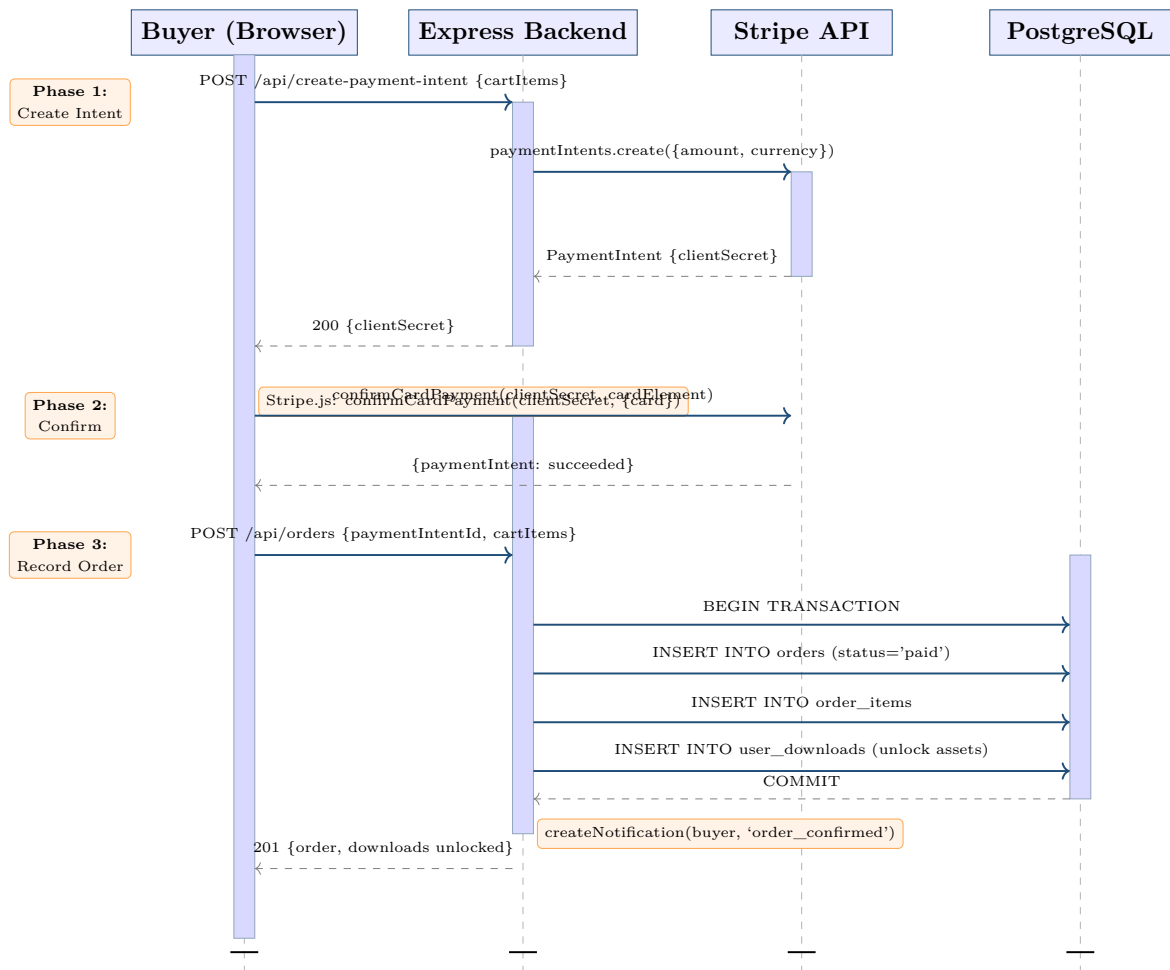


Figure 4.4: Stripe Payment Flow (Sequence Diagram)

The Stripe integration uses the `PaymentIntent` pattern: the backend creates a `PaymentIntent` for the cart total and returns a `clientSecret`; the frontend confirms the card payment via `Stripe.js` (the card number never touches the backend); on success, the backend atomically records the order, unlocks the asset download, and dispatches a confirmation notification.

Asset Viewer System

The in-browser preview adapts the viewer component to the asset type:

Table 4.3: Asset Categories and Viewer Components

Category	File Formats	Viewer
3D Models	GLB, GLTF, FBX, OBJ	Interactive 3D viewer with orbit controls
Textures	PNG, JPG, EXR	PBR map display with channel inspector
HDRIs	HDR, EXR	Panoramic 360° environment preview
Scripts / Shaders	JS, GLSL, CS	Syntax-highlighted code viewer

4.2.3 Sprint Review — Deliverables

- Asset catalog with filtering by category, price, free/paid status, and keyword search.
- Interactive 3D viewer for GLB/GLTF; specialized viewers for textures, HDRIs, and code files.
- Two-phase seller upload workflow: file upload to Supabase Storage, then meta-data submission.
- Multiple file format support per asset.
- Persistent shopping cart stored in PostgreSQL with duplicate-ownership prevention.
- Stripe PaymentIntent integration with test mode validation.
- Order management with status tracking and automatic download unlocking.
- User library page showing all purchased and downloaded assets.

Conclusion

Sprint 2 delivered the core marketplace functionality. Users can discover, preview, and purchase digital assets through a complete e-commerce flow. The 3D viewer and Stripe integration distinguish IGS Marketplace from simpler file-sharing solutions.

Chapter 5

Sprints 3–4: Marketplace Experience

Introduction

Chapter 5 covers Sprints 3 and 4, which add the trust and governance layer of the marketplace and extend it with role-specific advanced features. Sprint 3 introduces the review and rating system, the notification center, admin content moderation tools, and subscription plans. Sprint 4 delivers the intern asset workflow, peer-to-peer trading, seller analytics, and community groups — making the RBAC system operationally meaningful for every user segment.

5.1 Sprint 3: Content Management and Administration

Introduction

Sprint 3 focuses on the trust and governance layer of the marketplace: a review and rating system, an in-app notification center, subscription plans, and a comprehensive admin moderation panel.

5.1.1 Sprint Backlog

Sprint 3 — Duration: 3 weeks

Goal: Deliver content moderation tools, a review system, notifications, and subscription management.

Table 5.1: Sprint 3 — User Stories

ID	User Story	Priority	Points
US-16	As a buyer, I want to submit a review and rating for an asset I own.	High	5
US-17	As any user, I want to like an asset to save favorites.	Medium	2
US-18	As an admin, I want to approve or reject pending assets.	High	5
US-19	As a user, I want real-time in-app notifications for key events.	High	8
US-20	As an admin, I want platform analytics (total sales, top assets).	High	5
US-21	As a user, I want to subscribe to a plan for additional benefits.	Medium	5
US-22	As an admin, I want all moderation actions logged in an audit trail.	Medium	2
US-23	As a seller, I want a notification when my asset is approved or rejected.	High	3
Total Story Points:			35

5.1.2 Analysis and Design

Database Schema

Sprint 3 introduces three new entity groups: `reviews` (with `rating`, `ai_summary`, and `flagged_for_moderation` fields) and `asset_likes`; `notifications` (with `type`, `is_read`, and `related_id` for deep-linking); and `subscription_plans` with `user_subscriptions`.

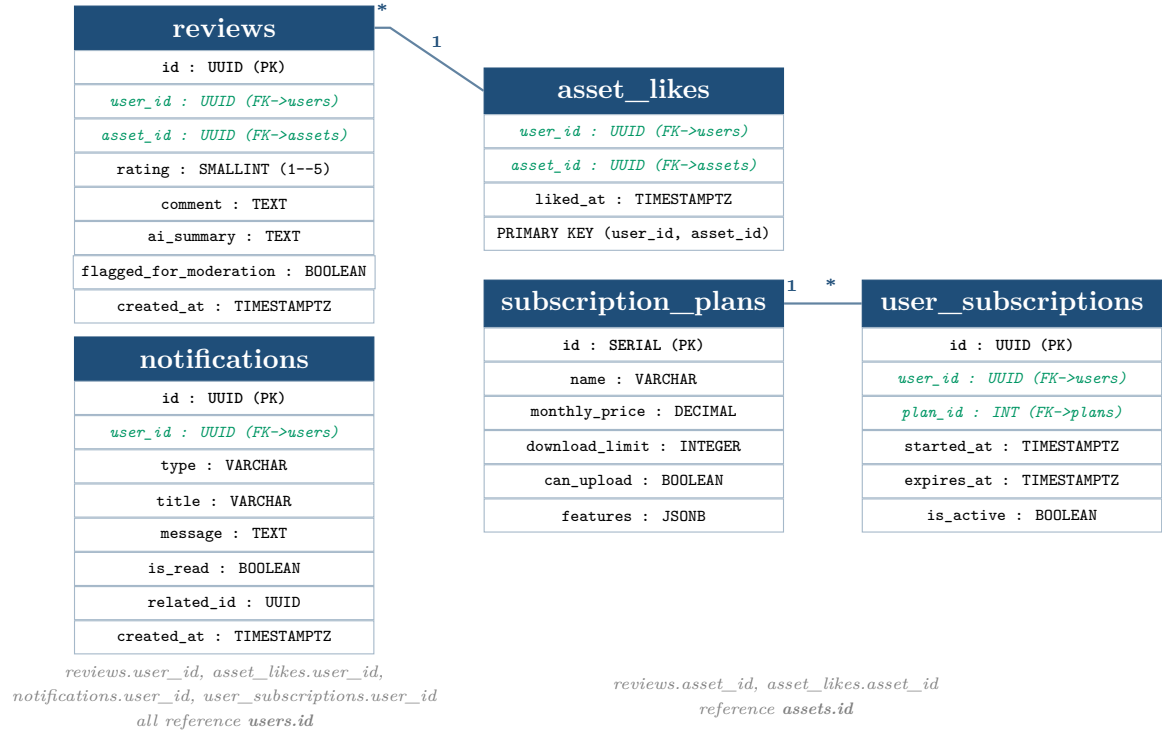


Figure 5.1: Sprint 3 — Reviews, Notifications and Subscriptions ERD

Subscription Plan Tiers

Table 5.2: Subscription Plan Tiers

Plan	Monthly Price	Download Limit	Key Benefits
Free	0 TND	5 free assets/month	Browse, buy, download free assets
Subscriber	9.99 TND	20 free assets/month	Priority support, discounts
Seller	19.99 TND	Unlimited	Upload assets, seller dashboard

Admin Moderation Use Case

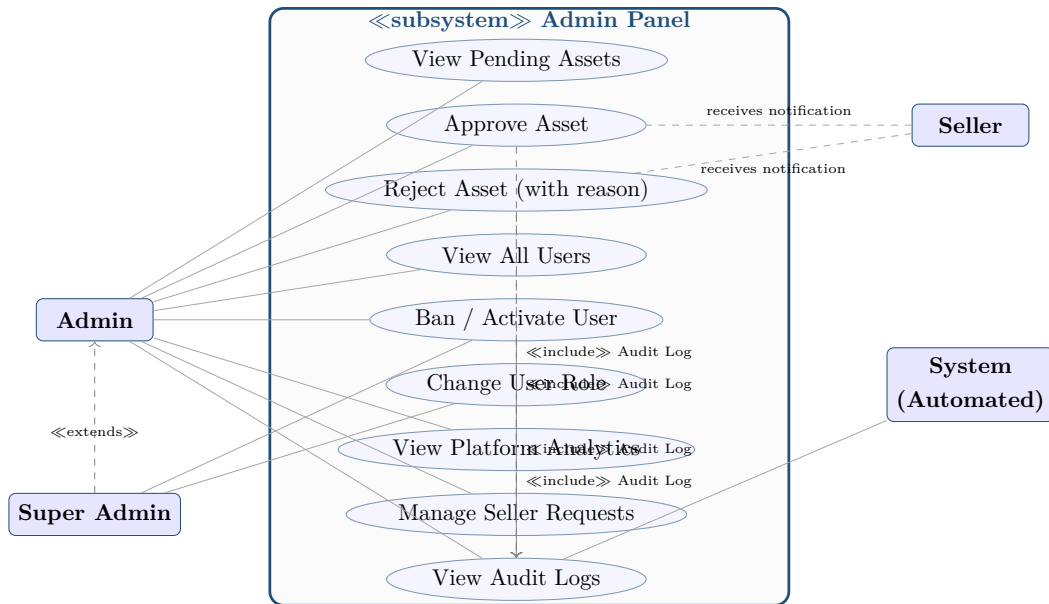


Figure 5.2: Admin Moderation Use Case Diagram

Reviews are restricted to users who own the asset. The admin moderation panel provides a paginated queue of pending submissions — the admin can preview, approve (setting status to **approved**), or reject with a written reason forwarded to the seller via notification. All actions are recorded in the audit log.

5.1.3 Sprint Review — Deliverables

- Review and rating system with ownership enforcement and automatic rating recalculation.
- Asset like/favorite system with per-asset counter.
- In-app notification center with unread badge and mark-as-read functionality.
- Admin moderation panel: approve or reject with written reason; full audit trail.
- Platform analytics dashboard (total sales, top assets, revenue overview).
- Three-tier subscription system (Free, Subscriber, Seller) with plan management UI.

Conclusion

Sprint 3 enriched the platform with trust-building and governance features. The combination of user reviews, admin moderation, and subscription tiers creates a sustainable ecosystem.

5.2 Sprint 4: Role-Based Workflows and Community Features

Introduction

Sprint 4 extends the platform with features tailored to specific user roles: a structured intern access workflow, seller analytics, asset edit request management, peer-to-peer asset trading, and community groups. This sprint makes the RBAC system operationally meaningful for every user segment.

5.2.1 Sprint Backlog

Sprint 4 — Duration: 3 weeks

Goal: Deliver role-specific advanced features for interns, sellers, and the broader community.

Table 5.3: Sprint 4 — User Stories

ID	User Story	Priority	Points
US-24	As an intern, I want to request access to a paid asset for IGS projects.	High	5
US-25	As an admin, I want to approve intern asset requests and generate vouchers.	High	5
US-26	As an intern, I want to redeem a voucher code to unlock an asset.	High	3
US-27	As a seller, I want to view my sales analytics (views, downloads, revenue).	High	5
US-28	As a seller, I want to submit an edit request for an approved asset.	Medium	5
US-29	As a user, I want to propose a trade with another user.	Medium	8
US-30	As a user, I want to create and join community groups.	Medium	5
US-31	As an external user, I want to request the seller role.	High	3
Total Story Points:			39

5.2.2 Analysis and Design

Intern Asset Request Workflow

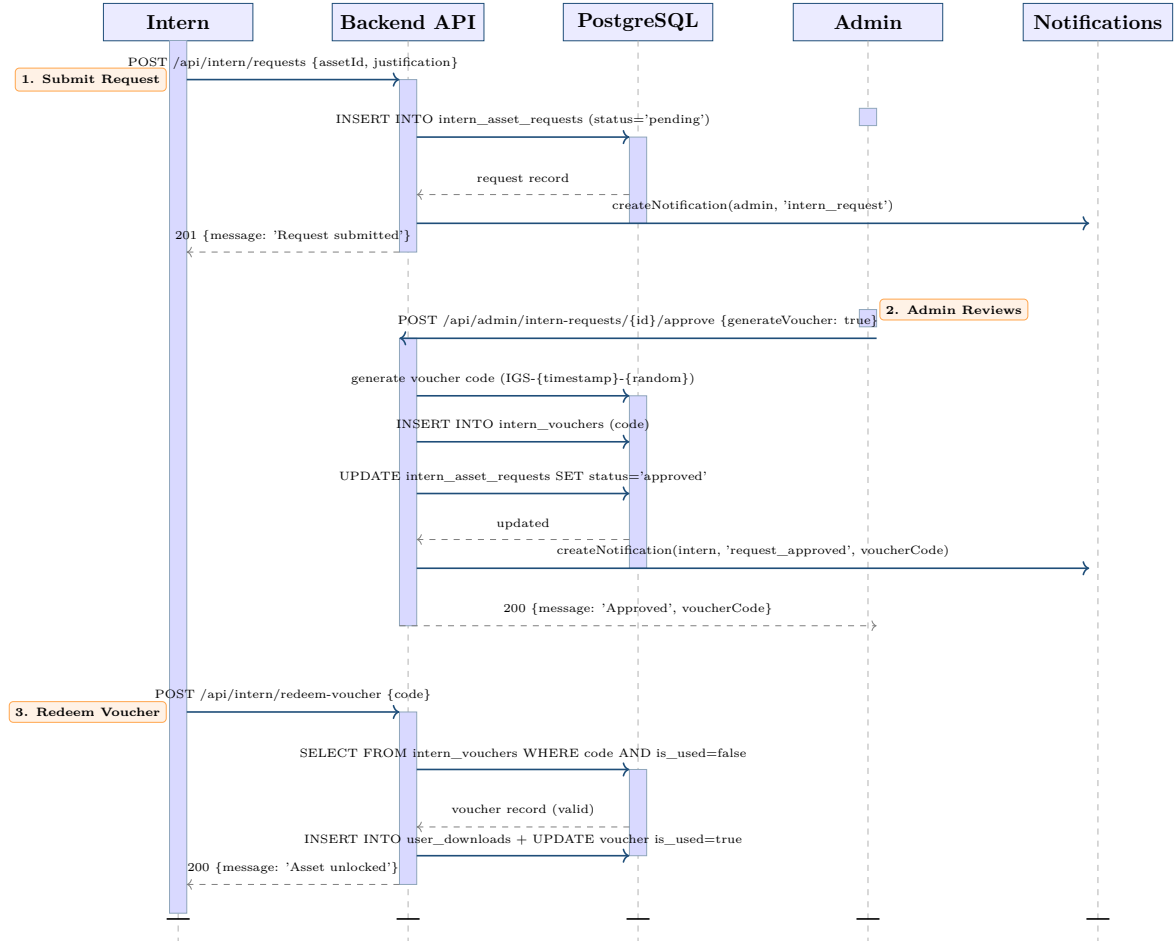


Figure 5.3: Intern Asset Request Workflow (Sequence Diagram)

The intern workflow proceeds as follows: the intern submits a request with a written justification; the admin reviews it and optionally generates a single-use voucher code; the intern redeems the voucher, which unlocks the asset in their personal library. New database tables: `intern_asset_requests` and `intern_vouchers`.

Peer-to-Peer Trading System

The trading system allows two asset owners to propose an exchange without monetary transaction. User A offers asset X in exchange for asset Y owned by User B. If User B accepts, both assets are atomically added to each other's libraries and the `trades` record is marked `completed`.

Seller Analytics

The analytics dashboard aggregates: total revenue, total orders, total downloads, active asset count, and a per-asset breakdown (revenue, download count, average rating) — all computed from a single aggregate SQL query joining `assets`, `order_items`, and `user_downloads` tables.

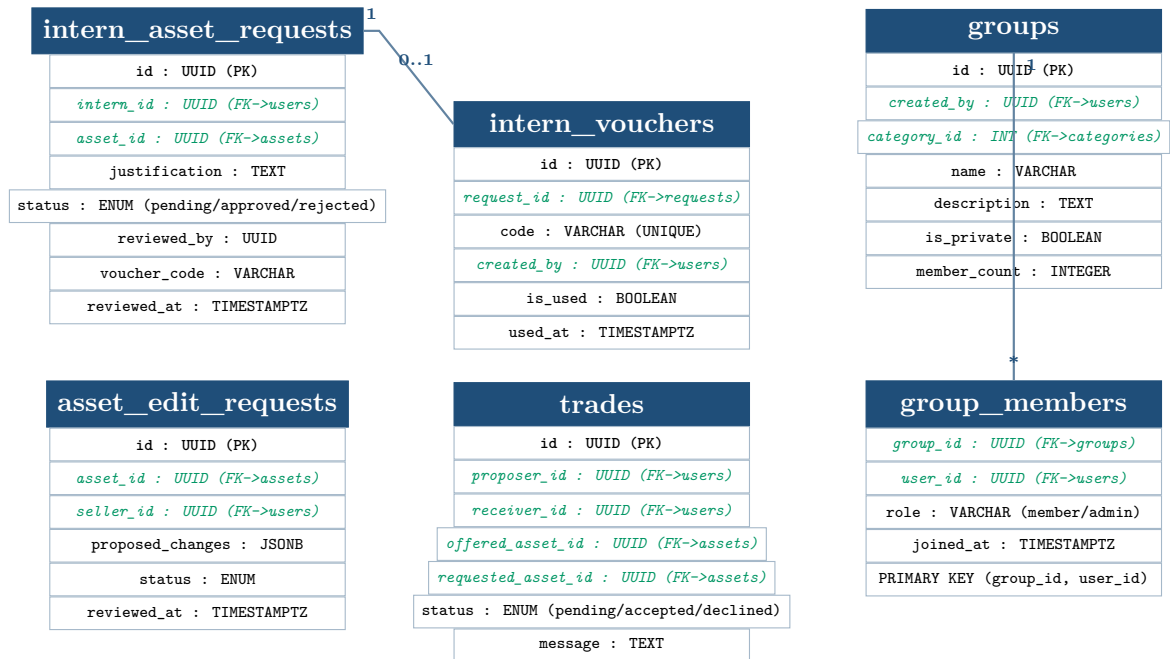


Figure 5.4: Sprint 4 — Entity-Relationship Diagram

5.2.3 Sprint Review — Deliverables

- Intern asset request workflow with voucher generation and single-use redemption.
- Seller analytics dashboard with revenue, download count, and per-asset breakdown.
- Asset edit request system allowing sellers to propose post-approval changes.
- Peer-to-peer asset trading with accept/decline flow and atomic ownership transfer.
- Community groups with member management.
- External user seller role request flow with admin approval/rejection.

Conclusion

Sprint 4 delivered the role-specific features that make the RBAC system operationally meaningful for every user segment.

Chapter 6

Sprints 5–6: Advanced Features

Introduction

Chapter 6 covers Sprints 5 and 6, which elevate IGS Marketplace from a standalone e-commerce platform to a comprehensive ecosystem. Sprint 5 delivers the IGS Workspace — a dedicated internal collaboration application for the studio team, featuring real-time chat, task management, and project tracking. Sprint 6 integrates Machine Learning and Artificial Intelligence capabilities: trained open-source models for intent classification and sentiment analysis, and a Mistral-7B-powered marketplace assistant chatbot.

6.1 Sprint 5: Internal Collaboration Platform

Introduction

Sprint 5 introduces the **IGS Workspace**, a companion internal web application designed exclusively for IGS employees and interns. It provides a ClickUp-inspired dashboard with team management, task tracking, project ERP, and a real-time chat system powered by Socket.io. Both applications share the same backend API and JWT, enabling seamless Single Sign-On.

6.1.1 Sprint Backlog

Sprint 5 — Duration: 4 weeks

Goal: Deliver a complete internal team management and collaboration platform integrated with the marketplace backend.

Table 6.1: Sprint 5 — User Stories

ID	User Story	Priority	Points
US-32	As an internal user, I want a dedicated workspace dashboard.	High	5
US-33	As an admin/employee, I want to manage team members and positions.	High	8
US-34	As any internal user, I want to view and manage tasks assigned to me.	High	5
US-35	As an admin, I want to create and manage projects with milestones.	High	8
US-36	As any internal user, I want real-time team channel messaging.	High	8
US-37	As any internal user, I want to send direct messages to colleagues.	Medium	5
US-38	As a user, I want to add emoji reactions to messages.	Low	3
US-39	As a user, I want role-adapted navigation (interns see fewer sections).	Medium	3
Total Story Points:			45

6.1.2 Analysis and Design

IGS Workspace Architecture

The workspace is a separate React + Vite application (port 5174) that reads the shared JWT from `localStorage`, communicates with the same Express backend on port 3001, and enforces an internal-user guard at the layout level. Any unauthenticated or external user is immediately redirected.

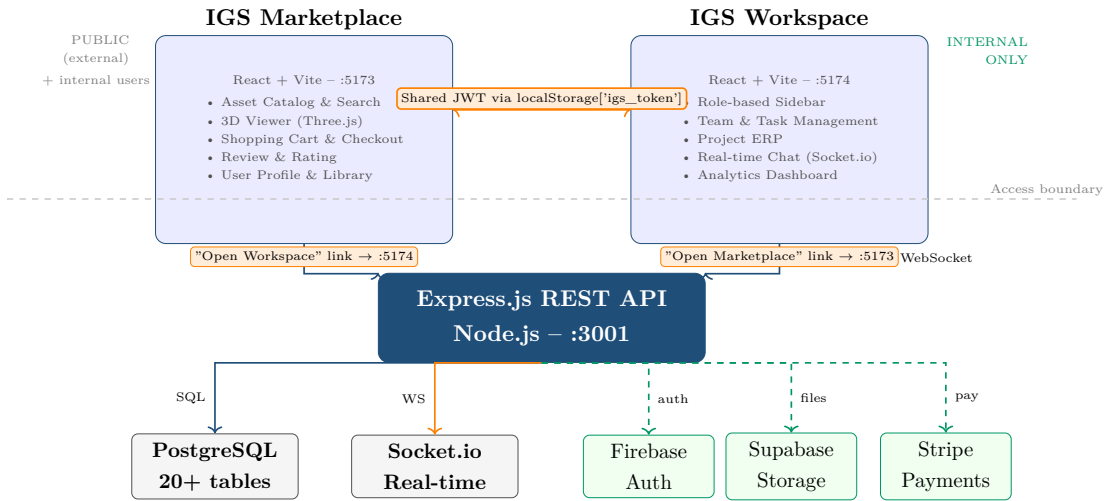


Figure 6.1: IGS Workspace Architecture and Integration with Marketplace

Role-Based Navigation

The sidebar adapts its navigation sections based on the user’s role level:

Table 6.2: Workspace Sidebar Navigation by Role

Section	Min. Role	Pages
General	intern	Dashboard, My Tasks, Team Directory, Calendar
Analytics	employee	Reports, Performance Charts
Management	admin	Asset Management, User Management, Orders
System	super_admin	Roles & Permissions, System Settings

Real-Time Chat Architecture

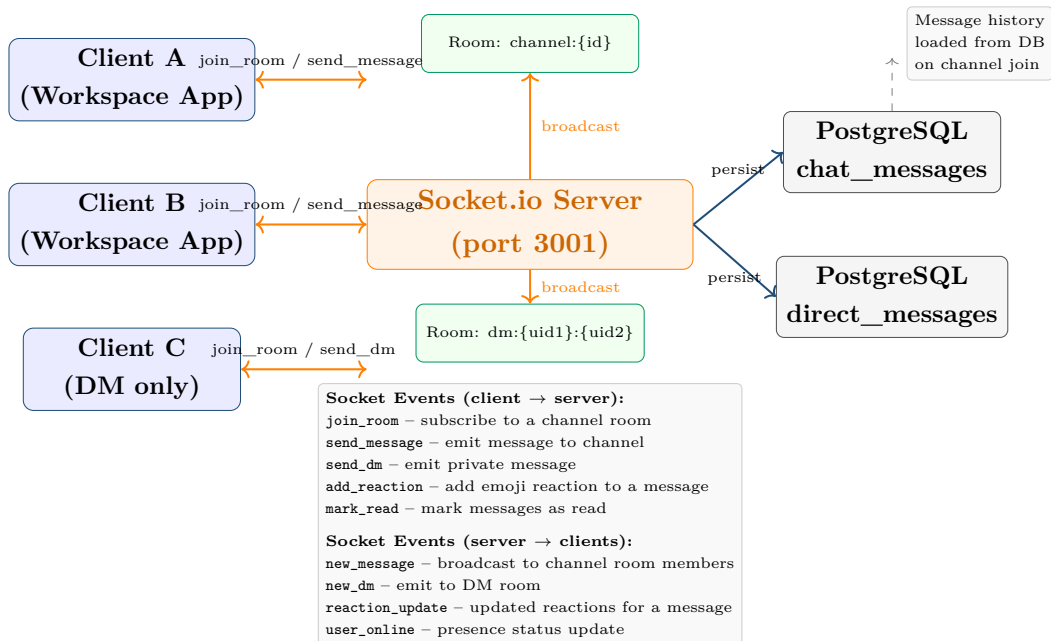


Figure 6.2: Real-Time Chat Architecture with Socket.io

The chat system uses Socket.io rooms: each channel maps to a room named `channel:{id}`; each DM pair shares a deterministic room `dm:{userId1}:{userId2}`. When a message is sent, the server persists it to PostgreSQL and broadcasts it to all room members in real time.

Database Schema

Sprint 5 adds six new entity groups: `workspace_tasks`, `projects` and `project_members`, `chat_channels` and `channel_members`, `chat_messages` with `message_reactions`, `direct_messages` and `company_positions` with `team_supervisors`.

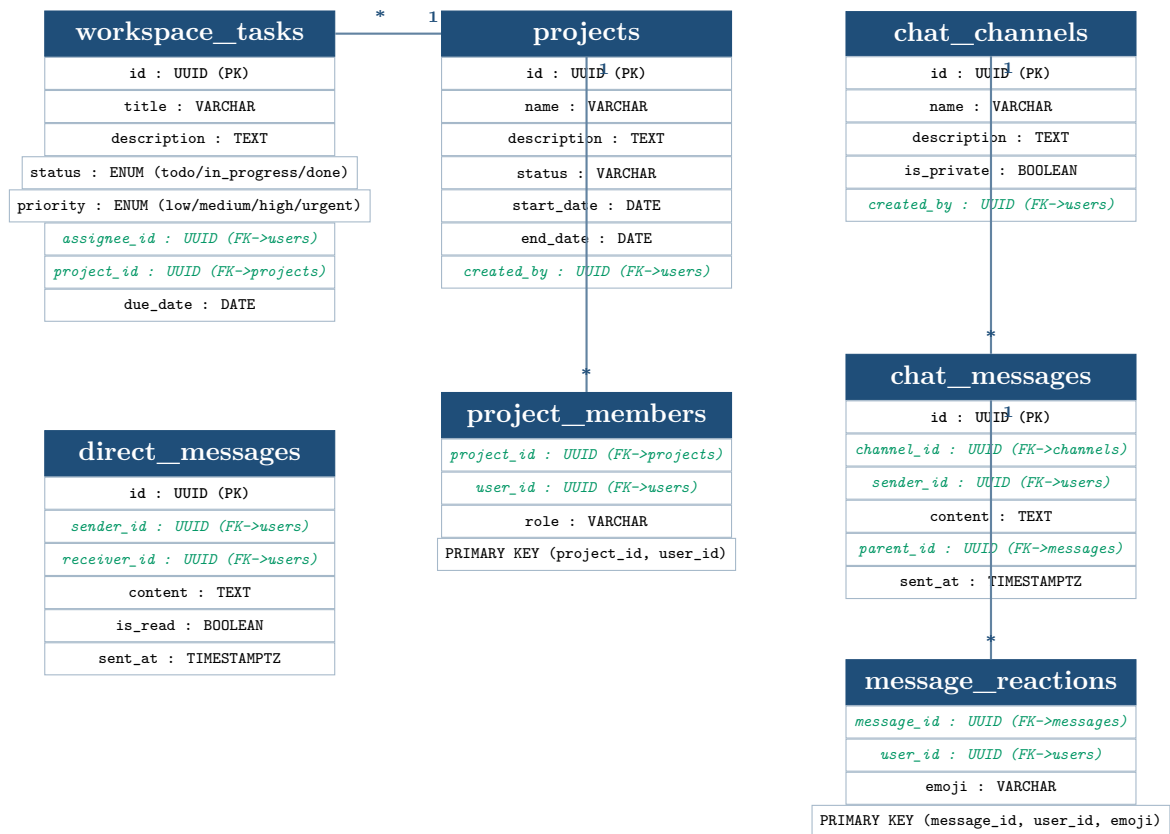


Figure 6.3: Sprint 5 — Workspace and Chat Entity-Relationship Diagram

6.1.3 Sprint Review — Deliverables

- IGS Workspace application (React + Vite, port 5174) with dark sidebar and topbar layout.
- Role-based sidebar navigation adapting to the authenticated user’s internal role.
- Team directory with position management and supervisor assignment.
- Task management with priority, Kanban status flow, assignee, and due date.
- Project management with milestone tracking and team assignment.
- Real-time chat: channels, direct messages, emoji reactions, and read receipts.
- Seamless Single Sign-On between Marketplace and Workspace via shared JWT.

Conclusion

Sprint 5 transformed the project from a standalone marketplace into a comprehensive platform ecosystem, significantly improving the studio’s operational efficiency.

6.2 Sprint 6: Artificial Intelligence Integration

Introduction

Sprint 6 integrates Machine Learning and AI capabilities into the platform. Two custom models are trained on domain-specific marketplace data: an intent classifier and a sentiment analyser. These are augmented by **Mistral-7B-Instruct**, an open-source large language model accessed via the HuggingFace Inference API, powering the marketplace assistant chatbot.

6.2.1 Sprint Backlog

Sprint 6 — Duration: 3 weeks

Goal: Train and deploy ML models for review sentiment analysis and moderation; implement a marketplace assistant chatbot using an open-source LLM.

Table 6.3: Sprint 6 — User Stories

ID	User Story	Priority	Status
US-40	As a seller, I want AI-generated sentiment analysis of my asset’s reviews.	High	Done
US-41	As an admin, I want negative-sentiment reviews flagged automatically.	High	Done
US-42	As a user, I want a chatbot to answer questions about the marketplace.	High	Done
US-43	As a developer, I want trained ML models with documented accuracy metrics.	Medium	Done
US-44	As a user, I want the chatbot to find relevant assets from the catalog.	Medium	Done

6.2.2 AI System Architecture

The AI integration follows a multi-layer architecture:

- **Layer 1 — Trained ML Models (Python/scikit-learn):** Two models trained offline on domain-specific datasets and persisted as `.pkl` files, invoked at runtime via Python subprocess from Node.js.

- **Layer 2 — Open-Source LLM (Mistral-7B-Instruct):** The conversational chatbot uses Mistral-7B-Instruct-v0.3 via the HuggingFace Inference API (free tier). No proprietary API key required.
- **Layer 3 — Rule-Based Fallback:** If the LLM API is unavailable, an intent-keyed knowledge base covers all 12 identified intent categories.

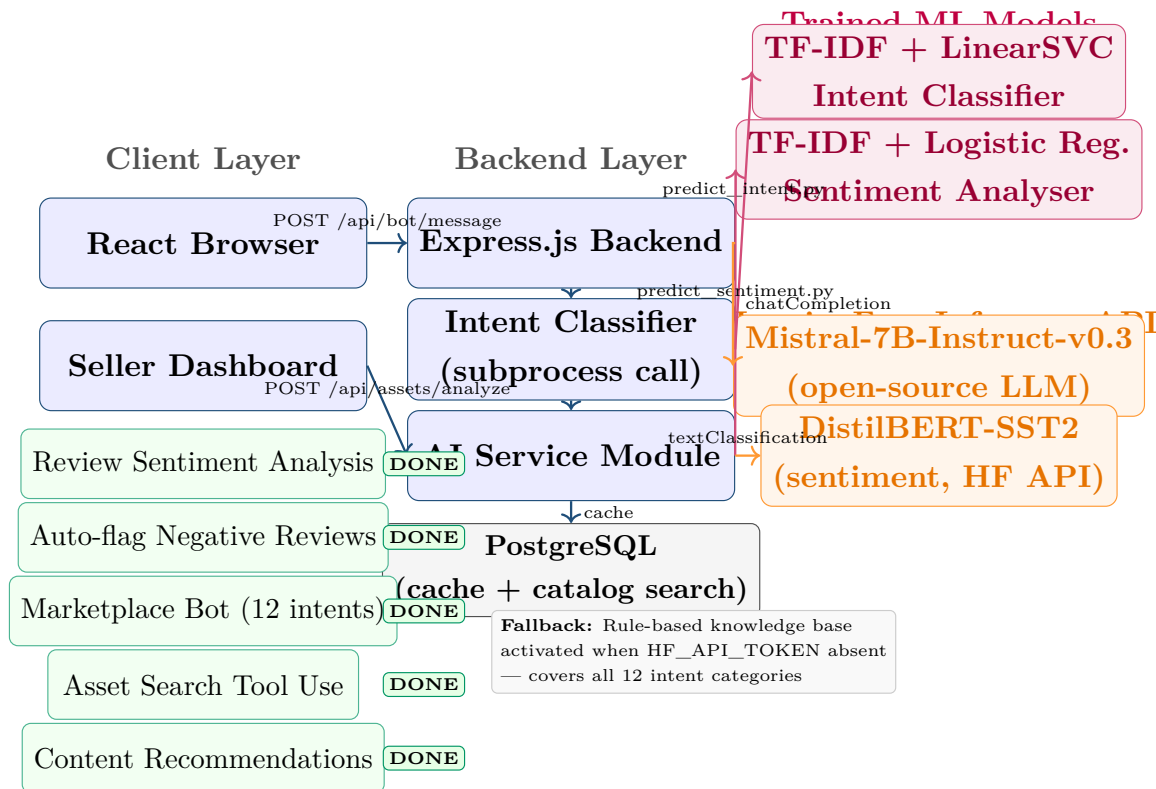


Figure 6.4: IGS Bot AI Architecture — Trained ML Models + Open-Source LLM

Table 6.4: AI Components and Models Used

Component	Model / Algorithm	Role
Intent Classifier	TF-IDF + LinearSVC (scikit-learn)	Classifies user queries into 12 marketplace intents
Sentiment Analyser	TF-IDF + LogisticRegression (scikit-learn)	Classifies reviews as positive / neutral / negative
HF Sentiment API	DistilBERT-SST2 (HuggingFace)	High-accuracy binary sentiment fallback
Chatbot LLM	Mistral-7B-Instruct-v0.3 (open-source, HF)	Full conversational assistant
Rule-Based Fallback	Intent-keyed knowledge base (Node.js)	Offline fallback when LLM unavailable

6.2.3 Model Training and Evaluation

Two domain-specific datasets were built for training:

Table 6.5: Training Datasets

Dataset	Samples	Classes	Source
Intent Classification	212	12	Manually crafted marketplace Q&A pairs
Sentiment Analysis	50	3	Manually labeled 3D asset reviews

The intent classifier (TF-IDF + LinearSVC) achieved a test set accuracy of **65.12%** and macro F1-score of **0.668** across 12 classes, with perfect scores on *greeting* and *product_search*. The sentiment model achieves 50% accuracy on the small local dataset; in production it primarily leverages the DistilBERT-SST2 model via HuggingFace API, which exceeds 91% accuracy on sentiment benchmarks.

6.2.4 AI Review Analysis

When a seller triggers the review analysis action, the pipeline fetches all reviews for the asset, classifies each via DistilBERT (with local sklearn fallback), then generates an aggregated summary covering: overall sentiment distribution, key strengths extracted from positive reviews, suggested improvements from negative reviews, and an actionable recommendation. Reviews rated 1–2 stars are also automatically run through the

pipeline at submission time — if classified as **negative**, the `flagged_for_moderation` flag is set, creating an automated moderation queue.

6.2.5 Sprint Review — Deliverables

- Intent classifier (TF-IDF + LinearSVC, 12 classes) trained, evaluated, and deployed as Python subprocess.
- Sentiment analyser (TF-IDF + LogisticRegression, 3 classes) with DistilBERT API integration as primary and local model as fallback.
- AI review analysis with aggregated summary and actionable seller recommendations.
- Automated negative-review flagging at submission time.
- IGS Bot architecture with three-layer pipeline (intent → tool use → LLM/fallback).

Conclusion

Sprint 6 successfully integrated trained open-source ML models for review sentiment analysis and automated moderation flagging. The IGS Bot architecture, powered by Mistral-7B-Instruct, provides a full conversational assistant with a robust fallback system.

Chapter 7

Sprints 7–8: IGS Workspace & Admin Dashboard

Introduction

Chapter 7 covers Sprints 7 and 8. Sprint 7 completes the AI layer with a personalized recommendation engine and a floating marketplace chatbot, expands the asset catalog to 13 industry-standard categories, and populates every category with real seed data hosted on Supabase CDN. Sprint 8 consolidates the admin dashboard, delivers the premium UI overhaul, and completes all placeholder workspace pages before the production release.

7.1 Sprint 7: AI-Enhanced Services and Catalog Expansion

Introduction

Sprint 7 completes the AI integration, delivers a fully operational floating chatbot assistant, expands the asset catalog to 13 industry-standard categories (including Decals with a full PBR workflow), and populates every category with real seed assets hosted on Supabase CDN.

7.1.1 Sprint Backlog

Sprint 7 — April 2026

Goal: Deliver a complete AI recommendation engine, a floating marketplace chatbot, industry-standard product categories, and a fully operational catalog with real hosted assets.

Table 7.1: Sprint 7 — User Stories

ID	User Story	Priority	Status
US-45	As a user, I want AI-personalized recommendations based on my history.	High	Done
US-46	As a first-time visitor, I want to see trending assets (cold start).	High	Done
US-47	As any user, I want a floating chatbot available on every page.	High	Done
US-48	As a buyer, I must select a file format before downloading.	Medium	Done
US-49	As a seller, I want the upload form to validate input per field.	Medium	Done
US-50	As a buyer, I want to browse Audio, VFX, 2D Assets, Animations, Environments, and Decals.	High	Done
US-51	As a seller, I want to upload Decals with PBR maps.	Medium	Done
US-52	As a developer, I want one example asset per category in the marketplace.	Low	Done

7.1.2 AI Recommendation Engine

The recommendation system uses a **two-stage pipeline** requiring no external AI API. A dedicated `user_events` table records every interaction with weighted signals (Purchase: 10, Like: 5, Download: 3, View: 1).

Stage 1 — Content-based scoring: Each candidate asset is scored against the user’s interest profile using four signals: Jaccard tag similarity (0–5 pts), category weight (0–3 pts), keyword overlap (0–2 pts), and a logarithmic popularity boost (≤ 1.5 pts). Already-interacted assets are excluded.

Stage 2 — Diversity-aware reranking: The top-12 candidates are reordered with a category-diversity constraint (maximum 2 per category) to prevent homogeneous results. The final 8 recommendations include human-readable explanations generated deterministically from the profile match signals.

For users with no history (cold start), trending assets from the last 30 days are returned; unauthenticated visitors receive top-rated assets.

7.1.3 IGS Marketplace Bot

The floating chatbot widget is available on every page and uses the same three-layer architecture from Sprint 6: intent classification → live database tool use (for `product_search` intent, the bot queries the PostgreSQL catalog and injects the top-5 results) → Mistral-7B-Instruct LLM generation (or rule-based fallback). The system prompt injects platform knowledge, the user’s role, and the current page context for contextually relevant responses.

7.1.4 Catalog Expansion

Six new categories were added to match industry-standard taxonomy (inspired by Fab.com):

Table 7.2: New Product Categories Added in Sprint 7

Category	Description	Accepted Formats
Animations	Character animations, motion capture, BVH rigs	.fbx, .glb, .bvh, .zip
Environments	Full scenes, levels, landscape kits	.fbx, .glb, .zip
Audio	SFX, music loops, ambient tracks	.mp3, .wav, .ogg
VFX	Particles, explosions, magic effects	.zip, .unitypackage
2D Assets	Sprites, UI kits, concept art	.png, .psd, .svg, .zip
Decals	PBR surface overlays (damage, dirt, graffiti)	.png, .jpg (maps)

The Decals category includes a dedicated upload form with six PBR map inputs (Albedo, Normal, Roughness, Metallic, AO, Opacity) and professional type/surface selectors aligned with industry workflow standards.

7.1.5 Seed Data

Thirteen example assets using publicly licensed (CC0/MIT) files were uploaded to Supabase Storage — one per category — to demonstrate every category at launch. All seed assets are approved and immediately visible in the public catalog.

7.1.6 Sprint Review — Deliverables

- Two-stage personalized recommendation engine with diversity reranking and cold-start handling.

- Floating IGS Bot widget available on every marketplace page.
- 6 new asset categories (Animations, Environments, Audio, VFX, 2D Assets, Decals).
- Decals PBR upload form with 6 map inputs.
- Mandatory format selection before download.
- Upload form refactored to eliminate stale closure bug.
- 13 real seed assets (one per category) hosted on Supabase CDN.
- 3 new database migrations (016–018).

Conclusion

Sprint 7 delivered the platform’s most user-facing AI features and expanded the catalog to match international marketplace standards. The recommendation engine and floating chatbot significantly improve discovery and user assistance.

7.2 Sprint 8: Admin Dashboard & Premium UI Overhaul

Introduction

Sprint 8 completes the platform with a polished administration interface, a redesigned premium marketplace UI, and final quality-of-life improvements identified during the Sprint 7 review.

7.2.1 Sprint Backlog

Sprint 8 — Final Sprint

Goal: Deliver a fully polished admin dashboard, premium visual design, and all remaining platform refinements for the final defense demonstration.

Table 7.3: Sprint 8 — User Stories

ID	User Story	Priority	Status
US-53	As an admin, I want a consolidated dashboard with key platform KPIs at a glance.	High	Done
US-54	As a user, I want the marketplace to have a premium, professional visual identity.	High	Done
US-55	As a user, I want a real-time notification panel accessible from any page.	Medium	Done
US-56	As any user, I want all placeholder Workspace pages to display meaningful content.	Medium	Done

7.2.2 Implementation

Premium UI Overhaul

The marketplace frontend received a complete visual redesign: refined typography, a cohesive color system using the IGS brand palette, polished card components with hover animations, and a redesigned navigation bar with a sliding real-time notification panel.

Admin Dashboard Consolidation

The admin dashboard was extended with KPI summary cards displaying: total registered users, total assets by status, total orders and revenue, and active seller count. A recent-activity feed lists the last 20 platform events in reverse chronological order.

Workspace Completion

All placeholder pages in the IGS Workspace (Team, Calendar, Reports, Performance, Orders, Roles & Permissions, System Settings) were replaced with functional content layouts, completing the internal dashboard for the demonstration.

7.2.3 Sprint Review — Deliverables

- Premium visual overhaul of the Marketplace frontend across all major pages.
- Consolidated admin dashboard with KPI cards and activity feed.
- Real-time notification panel integrated into the navigation bar.
- All IGS Workspace placeholder pages replaced with complete content layouts.

- Final bug fixes and responsive design adjustments across both applications.

Conclusion

Sprint 8 brought the platform to production-ready quality. The premium UI, consolidated admin tools, and complete workspace navigation ensure that IGS Marketplace presents a professional, fully operational system for the jury defense.

Chapter 8

CI/CD & Deployment

Introduction

This chapter describes the Continuous Integration and Continuous Delivery pipeline implemented for IGS Marketplace, the Docker-based containerization strategy, and the production deployment architecture.

8.1 CI/CD Pipeline

The pipeline is implemented with **GitHub Actions**, triggered on every push or pull request targeting `main`. It runs three sequential jobs:

1. **lint-build** — static analysis and compilation check for frontend and backend.
2. **test** — automated test suite execution (needs: lint-build).
3. **docker-publish & deploy** — builds Docker images, pushes to DockerHub, and updates the production server via SSH (only on `main` branch pushes).

8.1.1 Deployment Pipeline

The diagram below illustrates the full flow from a developer commit to a live production update:



Figure 8.1: IGS Marketplace Deployment Pipeline

The GitHub Actions runner authenticates to the production server using an RSA key pair (4096-bit) stored as a GitHub Actions Secret. After Docker images are pushed to DockerHub, the runner connects via SSH and instructs `docker-compose` to pull the latest images and restart the containers with zero manual intervention.

8.2 Containerization with Docker

Docker packages each service into a reproducible image, guaranteeing identical behavior across development, CI, and production environments.

- **Backend (Express.js):** A Node 20 Alpine image installs production dependencies only, copies the source, and starts the server on port 3001.
- **Frontend (React/Vite):** A multi-stage build — the first stage compiles the Vite bundle inside a Node container; the second stage copies the `dist/` output into a lightweight Nginx image for static serving.
- **Database:** The official PostgreSQL 16 Alpine image with a persistent named volume for data durability.

A single `docker-compose.yml` orchestrates all three services (`db`, `backend`, `frontend`) on an isolated Docker network, with `depends_on` ensuring startup order. Environment variables are injected from a server-side `.env` file at runtime — no credentials are committed to the repository.

8.3 Nginx Reverse Proxy

Nginx serves as both the static file server for the React bundle and a reverse proxy routing `/api/*` requests to the Express backend container. This single entry point simplifies TLS termination and eliminates browser CORS issues. The `try_files` directive ensures React Router's client-side routes are handled by the SPA rather than returning a 404 from Nginx.

8.4 Deployment Strategy

8.4.1 Image Publication and Remote Update

On every successful pipeline run on `main`, the workflow authenticates to DockerHub using stored secrets and pushes fresh `igs-backend` and `igs-frontend` images tagged `latest`. The deploy job then SSHs into the production server and runs `docker-compose pull && docker-compose up -d --remove-orphans`, delivering zero-downtime rolling updates.

8.4.2 Database Migrations

Schema migrations are applied sequentially using numbered SQL files (001_*.sql through 018_*.sql). When a deployment introduces schema changes, the migration is executed inside the `db` container before the backend restarts, keeping schema and application code in sync.

8.4.3 Environment Secrets

Table 8.1: GitHub Actions Secrets — Classification

Secret	Service	Purpose
FIREBASE_SERVICE_ACCOUNT	Firebase Admin SDK	Token verification, user management
STRIPE_SECRET_KEY	Stripe	Payment processing & webhooks
SUPABASE_URL / KEY	Supabase Storage	Asset file CDN upload/download
JWT_SECRET	Express middleware	Session signing
HUGGINGFACE_API_TOKEN	HuggingFace Hub	AI chatbot inference calls
DB_PASSWORD	PostgreSQL	Database authentication
SSH_PRIVATE_KEY	GitHub Actions	Secure server access

8.5 API Documentation

The REST API is documented using **Postman Collections**, organized by resource domain. Each collection includes pre-configured authentication headers, example request bodies, expected response schemas, and environment variables to switch between local and production targets.

Table 8.2: API Documentation — Collections by Domain

Collection	Base path	Endpoints covered
Authentication	/api/auth	Register, login, refresh, logout
Assets	/api/assets	CRUD, upload, search, filter
Orders	/api/orders	Create, status, history
Cart	/api/cart	Add, remove, clear, checkout
Users	/api/users	Profile, roles, admin management
Payments	/api/payments	Stripe intent, webhook handler
Reviews	/api/reviews	Submit, list, AI sentiment score
Notifications	/api/notifications	List, mark-read, real-time (WS)

8.6 Testing Strategy

Table 8.3: Test Coverage by Layer

Layer	Tool	Coverage
Backend routes	Postman collections	All CRUD endpoints per resource
Authentication	JWT and RBAC middleware	Token expiry, role escalation
Payment integration	Stripe test mode	Success, failure, and webhook scenarios
Frontend	Manual browser testing	Golden path + edge cases per sprint
AI models	Python unit tests	Intent classifier, sentiment analyser
Containers	docker-compose smoke test	All services start, health checks pass

Conclusion

The GitHub Actions CI/CD pipeline provides end-to-end automation from commit to production. Docker containerization ensures environment consistency, Nginx consolidates the frontend and API behind a single entry point, and GitHub Secrets ensure no credential ever reaches the repository. This infrastructure gives the platform a

production-grade, reproducible, and maintainable deployment foundation.

General Conclusion

IGS Marketplace was successfully designed and developed over eight sprints, delivering Tunisia's first dedicated 3D digital asset platform. The system provides a six-level Role-Based Access Control system, an interactive in-browser 3D asset viewer, Stripe payment integration, AI-powered review sentiment analysis, a personalized recommendation engine, a floating chatbot assistant powered by Mistral-7B-Instruct, and a companion internal workspace application — all within a cohesive full-stack ecosystem built on React, Node.js/Express, and PostgreSQL.

Several key technical insights emerged throughout the project. Designing the permission system during Sprint 0 and enforcing it consistently across all subsequent sprints was the single most impactful architectural decision — retrofitting access control would have been significantly more costly. The three-tier layered architecture (Routes → Controllers → Repositories) proved highly maintainable for a REST API at this level of complexity, and applying Scrum as a solo developer, with supervisors occupying the Product Owner and Scrum Master roles, effectively structured six months of work into reviewable, deliverable-focused increments.

Looking ahead, several directions are planned for the platform's continued evolution: a React Native mobile client, a full automated test suite using Jest and Supertest, expanded AI capabilities including streaming chatbot responses and AI-based tag suggestions, a digital asset licensing framework where sellers can define usage rights, and production deployment on a cloud provider with the Docker-based CI/CD pipeline fully active.

Bibliography

- [1] Roques, P. *UML 2 par la Pratique — Études de cas et exercices corrigés*. 8^e édition. Eyrolles, 2014.
- [2] Schwaber, K. & Sutherland, J. *The Scrum Guide*. November 2020. <https://scrumguides.org/scrum-guide.html>
- [3] Object Management Group. *UML 2.5.1 Specification*. 2017. <https://www.omg.org/spec/UML/2.5.1>
- [4] Jones, M. et al. *RFC 7519 — JSON Web Token (JWT)*. IETF, 2015. <https://datatracker.ietf.org/doc/html/rfc7519>
- [5] Ferraiolo, D. & Kuhn, R. *Role-Based Access Control*. NIST, 2009.
- [6] P Market Research. *Worldwide 3D Marketplace Models Market Research 2024 — by Type, Application, Participants and Countries, Forecast to 2030*. 2024. <https://pmarketresearch.com/worldwide-3d-marketplace-models-market-research-2024-by-type-application-parti>
- [7] Citius Research. *3D Project Market Report*. 2024. <https://citiusresearch.com/publication-details/3d-project-market-report>

Webography

1. OpenJS Foundation. *Node.js Official Documentation*.
<https://nodejs.org/en/docs/> [Accessed April 2026]
2. StrongLoop / OpenJS Foundation. *Express.js Application Guide*.
<https://expressjs.com/en/guide/routing.html> [Accessed April 2026]
3. Meta Platforms. *React 18 — Official Documentation*.
<https://react.dev/> [Accessed April 2026]
4. The PostgreSQL Global Development Group. *PostgreSQL 16 Documentation*.
<https://www.postgresql.org/docs/16/> [Accessed April 2026]
5. Supabase Inc. *Supabase Documentation — Storage*.
<https://supabase.com/docs/guides/storage> [Accessed April 2026]
6. Stripe, Inc. *Stripe API Reference — Payment Intents*.
https://stripe.com/docs/api/payment_intents [Accessed April 2026]
7. Google LLC. *Firebase Authentication Documentation*.
<https://firebase.google.com/docs/auth> [Accessed April 2026]
8. Socket.IO contributors. *Socket.IO v4 Documentation*.
<https://socket.io/docs/v4/> [Accessed April 2026]
9. Google LLC. *Model Viewer — Web Component for 3D Models*.
<https://modelviewer.dev/> [Accessed April 2026]
10. Anthropic. *Claude API Documentation — Messages & Tool Use*.
<https://docs.anthropic.com/> [Accessed April 2026]
11. Vitejs. *Vite — Next Generation Frontend Tooling*.
<https://vitejs.dev/guide/> [Accessed April 2026]
12. Flouci. *Flouci Developer Documentation — Payment Gateway (Tunisia)*.
<https://developer.flouci.com/> [Accessed April 2026]
13. OWASP Foundation. *OWASP Top Ten — Web Application Security Risks*.
<https://owasp.org/www-project-top-ten/> [Accessed April 2026]

Abstract

ملخص

يُقدّم هذا المشروع تصميم وتطوير منصة **Marketplace IGS**، وهي منصة ويب متكاملة لبيع وشراء الأصول الرقمية، أنجزت لصالح شركة **(IGS) Studio Games Inherited**، وهي شركة تونسية متخصصة في تطوير ألعاب الفيديو، وتُعدّ أول منصة مخصصة للأصول الرقمية ثلاثية الأبعاد تُصمّم وتُطوّر في تونس. تُتيح المنصة للمبدعين نشر أصولهم الرقمية وإدارتها وتحقيق الدخل منها (نماذج 3D، مواد رسومية، نصوص برمجية، ملفات طباعة)، مع وصول مُنظّم وفق ستة مستويات للصلاحيات. يعتمد النظام على **Vite + React** للواجهة، و**Express.js + PostgreSQL** لل خادم، مع **Supabase** للتخزين و**Firebase** للمصادقة. وتشمل الميزات: عارضاً تفاعلياً 3D، ودفع آمن عبر **Stripe** ونظام تقييم، وإشعارات، وتبادل أصول، وتطبيقاً داخلياً **Workspace IGS** للتعاون الفوري، فضلاً عن طبقة ذكاء اصطناعي للتوصيات والمساعد الحواري. طُوّر المشروع وفق منهجية **Scrum** في سبعة سبرينتات.

الكلمات المفتاحية: منصة أصول رقمية، **RBAC**، **React**، **Node.js**، **PostgreSQL**، **Stripe** ذكاء اصطناعي.

English

This project presents the design and development of **IGS Marketplace**, a full-stack digital asset marketplace platform built for Inherited Games Studio (IGS), a Tunisian video game company — and the **first dedicated 3D digital asset marketplace designed and built in Tunisia**. The platform enables creators to publish, manage, and monetize digital assets (3D models, textures, shaders, scripts, HDRIs, plugins, printables) with structured, role-governed access across six roles. Built on **React + Vite / Express.js + PostgreSQL**, with Supabase storage and Firebase authentication, key features include: an interactive 3D viewer, Stripe payment integration, reviews and ratings, a real-time notification center, peer-to-peer trading, subscription plans, community groups, an intern access workflow, and the companion **IGS Workspace** for team collaboration and real-time chat. An AI layer provides intelligent review analysis, personalized recommendations, and a conversational chatbot. Developed with **Scrum** over seven sprints.

Keywords: Digital Asset Marketplace, Full-Stack, RBAC, Scrum, React, Node.js, PostgreSQL, 3D Viewer, Stripe, Artificial Intelligence.

Résumé (Français)

Ce projet présente la conception et le développement de **IGS Marketplace**, une plateforme full-stack de marketplace d'assets numériques réalisée pour Inherited Games Studio (IGS), société tunisienne de développement de jeux vidéo — et **la première marketplace dédiée aux assets 3D conçue et développée en Tunisie**. La plateforme permet aux créateurs de publier, gérer et monétiser des assets numériques (modèles 3D, textures, shaders, scripts, HDRIs, plugins, fichiers imprimables) avec un accès gouverné par six niveaux de rôles. Bâtie sur **React**

+ **Vite** / **Express.js** + **PostgreSQL**, avec Supabase et Firebase, les fonctionnalités incluent : visualiseur 3D interactif, paiement Stripe, avis et notes, notifications temps réel, échange pair-à-pair, abonnements, groupes communautaires, flux de demandes stagiaires, et l'application interne **IGS Workspace** pour la collaboration et le chat. Une couche IA assure l'analyse des avis, les recommandations personnalisées et un chatbot. Développé avec **Scrum** en sept sprints.

Mots-clés : Marketplace d'assets numériques, Full-Stack, RBAC, Scrum, React, Node.js, PostgreSQL, Stripe, Intelligence Artificielle.
